A Classical Reasoning Agent for Logical Entailment

Siddharth Bajpai - M26MAC009
AI Assignment

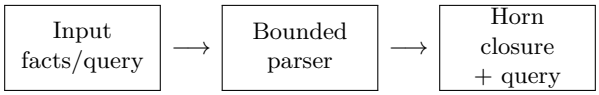


Figure 1. System pipeline. Unsupported input is reported at parsing rather than passed to inference as an assumed fact or label.

Abstract

We implement a symbolic system for deciding whether a query follows from facts and rules. The inference core represents signed atoms and Horn clauses, uses first-order unification, and applies forward chaining to a fixed point. Natural-language and formal-logic inputs are handled by bounded parsers; unsupported statements are reported rather than silently converted into negative predictions. On the loaded public splits, the system supports all 250 RuleTaker examples (97.6% accuracy), 55 of 203 FOLIO examples (83.3% accuracy on the supported, labeled subset), and none of the 1,200 CounterBench examples. These results emphasize the distinction between reasoning accuracy and input coverage.

1. Introduction

The task is to determine whether a query is entailed by a knowledge base, not to predict its label with a generative model. Our design separates data loading, input parsing, and the classical reasoning engine. This makes the inference procedure inspectable and exposes a central limitation: a sound reasoner cannot compensate for a parser that does not represent the input's logic.

2. Representation and Inference

2.1. Representation

A fact is a signed atom $P(t_1, \ldots, t_n)$, where terms are constants or variables. A rule is a conjunction of body atoms implying one head atom. For example,

$$\textit{young}(x) \rightarrow \textit{smart}(x),$$
$$\textit{smart}(x) \wedge \textit{kind}(x) \rightarrow \textit{successful}(x).$$

RuleTaker's controlled-English forms are mapped to this representation. For FOLIO, the adapter uses the provided formal premise and conclusion columns when they fit the supported Horn fragment. Positive existential conjunctions are represented using fresh witness constants. Disjunction, exclusive-or, nested quantifiers, and other unsupported formulas are marked unsupported.

2.2. What the Engine Computes

The representation is a fragment of first-order logic chosen to make inference explicit. A ground fact contains no variables; a rule variable is matched consistently across every atom in that rule. For example, matching $\textit{young}(x)$ against $\textit{young}$(Bob) binds x to Bob. The second body atom must then match $\textit{kind}$(Bob), rather than an unrelated individual. Only when every body atom has a match is the instantiated head added.

Forward chaining repeats this process until a complete pass adds nothing. The result is a least fixed-point closure for the encoded positive Horn rules: every derived atom has a finite chain of rule applications from the initial facts. The returned trace records such applications and can be inspected to explain a positive answer. This is not a general first-order theorem prover. It does not search arbitrary formula structures, reason by contradiction, or expand the rule language when a premise cannot be parsed.

2.3. Classical Reasoning

The independent module logic_engine.py implements unification and forward chaining. For each rule, unification matches its body against known ground facts and produces substitutions. The instantiated head is added to the knowledge base; iteration stops at a fixed point. Query entailment is checked by membership in the derived closure. The engine is deterministic and does not use a learned model to choose labels.

Dataset	Rows	Supported	Coverage	Accuracy
RuleTaker	250	250	100.0%	97.6%
CounterBench	1,200	0	0.0%	–
FOLIO	203	55	27.1%	83.3%

Table 1. Latest recorded public-split results. FOLIO accuracy is only on supported rows with binary labels, not the full split.



Figure 2. Input coverage on the loaded splits. Bar lengths are proportional to coverage (maximum 100%); accuracy is not shown because it is measured on a different denominator.

2.4. Reasoning Trace

Given facts *young*(Bob) and *kind*(Bob), the first rule yields *smart*(Bob). The second rule then yields *successful*(Bob). Thus the query "Bob is successful" is supported by a two-step derivation:

$$\textit{young}(\text{Bob}) \Rightarrow \textit{smart}(\text{Bob}) \Rightarrow \textit{successful}(\text{Bob}).$$

3. Experiments

We loaded the public RuleTaker test split (250 examples), two CounterBench JSON files (1,200 examples total), and FOLIO validation (203 examples). No private evaluation data was used. Accuracy is computed only for parsed examples with a recognizable binary gold label; coverage is the fraction of all loaded rows handled by the system. Inference time includes parsing and reasoning for each split and is environment-dependent.

The measured inference times were 0.238 seconds for RuleTaker, 0.181 seconds for CounterBench, and 0.072 seconds for FOLIO. These are single-run timings, not hardware-independent benchmarks. RuleTaker produced six incorrect predictions despite full parser coverage; a detailed error taxonomy was not part of this run.

3.1. Evaluation Protocol

Each dataset row is sent through the same parse-and-predict path. A row is supported only when its context and query are accepted and the reasoner returns a prediction. Coverage is the number of supported rows divided by the total rows loaded. Accuracy uses only supported rows that also have a recognized binary gold label:

$$\text{coverage} = \frac{N_{\text{supported}}}{N_{\text{all}}}, \qquad \text{accuracy} = \frac{N_{\text{correct}}}{N_{\text{supported, labeled}}}.$$

This makes the denominator explicit: an unsupported row contributes to the coverage denominator, but not the accuracy denominator. A dash in the CounterBench accuracy column means that there are no supported labeled rows from which to calculate accuracy; it does not mean that every prediction was wrong.

The results refer to the public RuleTaker test split, FOLIO validation split, and two loaded CounterBench files only. They are a bounded experiment, not evidence about held-out private data or other versions of the datasets. Timings are the recorded wall-clock time for processing each complete split, including parsing and inference; they are single runs and should not be compared as hardware-independent throughput measurements. The notebook also exercises a small synthetic example as a smoke test. Such a check can verify basic parser, inference, and output behavior, but cannot substitute for benchmark evaluation.

For reproducibility and diagnosis, the batch evaluator retains per-row predictions, labels, statuses, and traces. This is more informative than a single accuracy number: a trace can show which rule produced an answer, while an explicit unsupported status identifies a coverage failure before inference. However, the recorded summary output is not yet a complete audit artifact. In particular, the aggregate results do not include a reconciled row-level error list for the six RuleTaker mistakes, and the separately printed FOLIO status diagnostic differs from the scored support count. Consequently the table reports the recorded metrics, while stronger claims about the precise causes of individual errors await a single rerun that exports and cross-checks all row-level fields.

4. Failure Analysis and Limitations

4.1. Why Coverage Is Limited

The largest observed issue is a mismatch between benchmark language and parser scope. A natural-language parser must identify predicates, arguments, quantifier scope, negation, and connective structure. The current controlled-English parser recognizes only a limited set of sentence templates. A sentence can therefore be logically simple yet unsupported if it uses an unrecognized phrase, relative clause, or surface construction. This is a parsing failure: the reasoner never receives a faithful logical representation.

The FOLIO adapter avoids some language ambiguity by reading formal formulas, but the adapter still

accepts only a subset. Its supported cases include literals, conjunctions, Horn implications, universal rules in the supported form, and restricted positive existential premises. Disjunction and exclusive-or are not Horn clauses; nested or unsupported quantifier patterns also fall outside the implementation. Such formulas cannot safely be flattened into independent facts or rules: doing so can change what follows from the premises. Refusing them is preferable to silently producing a plausible but unsound answer. The recorded FOLIO accuracy (83.3%) applies to the 55 supported examples with usable labels, not to the 203-row split.

CounterBench has two distinct compatibility problems. First, its contexts use linguistic and structural patterns the text parser does not cover. The notebook's diagnostic output includes quantified, relative, or disjunctive constructions, as well as unsupported clauses and causal-language templates. Second, the benchmark asks about interventions and counterfactuals. A rule such as $A \rightarrow B$ says that A implies B in the encoded knowledge base; it does not specify what would happen if an external intervention forced A to change. Counterfactual reasoning requires a causal model and intervention semantics, neither of which is represented by ordinary Horn closure. All 1,200 rows are therefore unsupported, so no CounterBench accuracy is available. The reported 0.181 seconds is the time to process the split and report that status, not evidence that the benchmark's reasoning problem was solved.

4.2. Why Accuracy and Labels Need Care

Coverage and accuracy answer different questions. Coverage is supported rows divided by all rows. Accuracy is correct predictions divided only by supported rows with recognizable binary labels. A system can consequently have high conditional accuracy and still fail to address most examples. For FOLIO, reporting 83.3% without its 27.1% coverage would overstate what the experiment establishes. For CounterBench, accuracy is undefined rather than zero: there are no supported examples in its denominator.

The benchmark interface also compresses entailment into True or False. The engine can establish True when the query matches a derived fact. An unproved query is emitted as False to fit the binary interface, but under open-world semantics that means only "not established from these premises." It does not establish that the negation is true. This distinction matters especially when knowledge is incomplete. A richer evaluation would distinguish entailed, contradicted, unknown, and unsupported cases, while retaining a mapping to the benchmark's required labels.

There is a second limitation in the evidence available for explaining errors. The recorded RuleTaker run has six wrong predictions, but it does not include a per-error taxonomy. It is therefore not possible to attribute those six errors to one specific parser rule or inference behavior from the aggregate metrics alone. The notebook also prints unsupported-reason diagnostics for FOLIO and CounterBench; those diagnostic status totals do not match the scored coverage totals reported elsewhere in its outputs. The reason strings are useful examples of parser boundaries, but they should not be presented as a verified breakdown of every excluded scored row. Resolving this requires rerunning one consistent evaluation path that saves each row's parse status, prediction, gold label, and failure reason together.

4.3. Error Sources and Next Checks

These distinctions give a practical order for addressing the issues:

1. Measure parser coverage by construction. Save the exact unsupported formula and a stable reason code for each row, then group failures by connective, quantifier, and language template. This identifies whether the dominant gap is syntax, semantics, or both.
2. Audit supported predictions. For each incorrect RuleTaker example, compare the parsed premises and query with the source text, then inspect the derived closure and trace. A parse mismatch points to the adapter; a faithful parse with a wrong conclusion points to inference or label interpretation.
3. Keep unsupported distinct from False. Preserve an explicit unsupported status throughout evaluation and submission generation. Do not silently coerce parse failures to a negative answer.
4. Use benchmark-specific semantics where needed. Improving CounterBench requires an intervention-aware model, not just more Horn rules. Expanding FOLIO requires a formally specified and tested extension for its missing connectives and quantifiers.

These checks do not assume that broader parsing automatically improves accuracy. Adding a connective without preserving its semantics can increase coverage while making predictions less reliable.

5. Conclusion

The implementation demonstrates classical unification and forward chaining with inspectable derivation traces. Public results show strong coverage on the loaded RuleTaker split, limited coverage on FOLIO, and a clear semantic mismatch with CounterBench. The next improvement should target representation

coverage and explicit error analysis rather than adding search algorithms without addressing those input and logic boundaries.

References

[1] RuleTaker dataset, Hugging Face. https://huggingface.co/datasets/jise/ruletaker

[2] CounterBench dataset, Hugging Face. https://huggingface.co/datasets/CounterBench/CounterBench

[3] FOLIO dataset, Hugging Face. https://huggingface.co/datasets/yale-nlp/FOLIO